

Compolsory Assignment 4: Variational Autoencoders

Please fill out the the group name, number, members and optionally the name below.

Group number: 26

Group member 1: Milad Tootoonchi

Group member 2: Makka Dulgaeva

Group member 2: Sania Minaipour

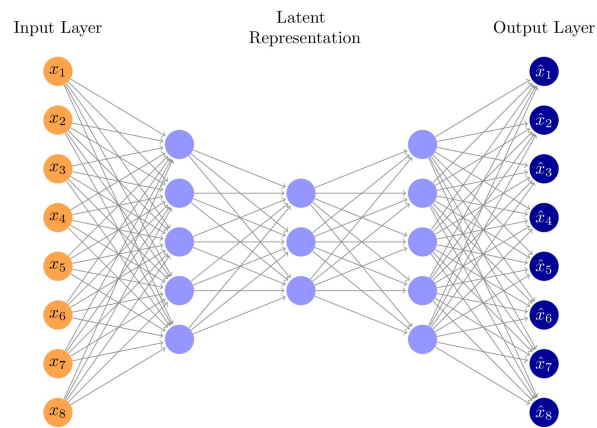
Assignment Submission

To complete this assignment answer the relevant questions in this notebook and write the code required to implement the relevant models. It consist of **two tasks**. At the end of each task, there are discussion questions that *must* be answered. The assignemnt is submitted by handing in this notebook as an .ipynb file and as a .pdf file.

We will explore autoencoder networks using the **Sign Language MNIST** dataset from Kaggle (as same as CA1). It contains 28×28 grayscale images of hand signs representing letters A–Y (excluding J and Z), for 24 classes in total. The data is provided as CSV files with a label column and 784 pixel columns. The standard split includes 27,455 training images and 7,172 test images

Autoencoder is an introduction to encoder-decoder architecture. Here we will use an encoder to transform the data into an latent representation, and a decoder to project it back into the visual space. Two main components: an **encoder** and a **decoder**.

- The encoder takes in the input data and maps it to a lower-dimensional latent representation. It typically consists of multiple layers that gradually reduce the dimensionality of the input data, capturing its essential features. The output of the encoder is a compressed representation of the input, often referred to as a code or latent vector.
- The decoder, on the other hand, takes the code from the encoder and reconstructs the original data from it. It mirrors the architecture of the encoder by gradually expanding the code back to the original dimensionality. The output of the decoder is a reconstruction of the input data, which ideally should closely resemble the original input.



Task 1

- Load the SignMNIST dataset.
 - Use the provided CSV files `sign_mnist_train.csv` and `sign_mnist_test.csv`.
 - Normalize the images to have pixel values between 0 and 1.
- Build an autoencoder model.
 - The encoder should compress the images into a lower-dimensional latent space.
 - The decoder should reconstruct the images from the latent space.
 - The intermediate layers can either be linear or convolutional.
- Train the autoencoder.
 - Use `binary-cross-entropy` loss function for *reconstruction* (i.e. target value = input value)
 - You can choose optimizer freely (Adam is recommended).
- Visualize the reconstructed images.
 - Compare original and reconstructed images.
- Visualize the latent space
 - Use PCA to visualize the 2d-latent space.
 - Use the digit as labels
- Answer the discussion questions

```
In [4]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [2]: # Import necessary Libraries
import pandas as pd
import numpy as np
import tensorflow.keras as ks
from tensorflow.keras import models, layers
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
from tensorflow.keras.callbacks import EarlyStopping
```

```
In [3]: # 1. Load and preprocess the Sign MNIST dataset
train = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/CA4/sign_mnist_train.csv")
X_train = train.drop(columns = ["label"]).to_numpy(dtype = np.float32)

test = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/CA4/sign_mnist_test.csv")
X_test = test.drop(columns = ["label"]).to_numpy(dtype = np.float32)

# Preprocessing
X_train = X_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
X_test = X_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0
```

```
In [4]: # 2. Build the autoencoder model
# Encoder
encoder = models.Sequential([
    layers.Input(shape = (28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation = 'relu', padding = 'same'),
    layers.MaxPooling2D((2, 2), padding = 'same'),
    layers.Conv2D(64, (3, 3), activation = 'relu', padding = 'same'),
    layers.MaxPooling2D((2, 2), padding = 'same')
])

# Decoder
decoder = models.Sequential([
    layers.Conv2DTranspose(64, (3, 3), strides = 2, activation = 'relu', padding = 'same'),
    layers.Conv2DTranspose(32, (3, 3), strides = 2, activation = 'relu', padding = 'same'),
    layers.Conv2D(1, (3, 3), activation = 'sigmoid', padding = 'same')
])

autoencoder = models.Sequential([encoder, decoder])
```

```
In [5]: # 3. Compile and train the autoencoder
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience = 5,
    min_delta = 0.001,
    restore_best_weights = True
)

autoencoder.compile(optimizer = 'adam', loss = 'binary_crossentropy')

history = autoencoder.fit(
    X_train, X_train,
    epochs = 20,
    batch_size = 128,
    shuffle = True,
    validation_data = (X_test, X_test),
    callbacks=[early_stopping]
)
```

```

Epoch 1/20
215/215 ————— 11s 26ms/step - loss: 0.6330 - val_loss: 0.5862
Epoch 2/20
215/215 ————— 2s 10ms/step - loss: 0.5896 - val_loss: 0.5832
Epoch 3/20
215/215 ————— 2s 11ms/step - loss: 0.5872 - val_loss: 0.5820
Epoch 4/20
215/215 ————— 2s 9ms/step - loss: 0.5856 - val_loss: 0.5812
Epoch 5/20
215/215 ————— 2s 9ms/step - loss: 0.5847 - val_loss: 0.5805
Epoch 6/20
215/215 ————— 2s 9ms/step - loss: 0.5847 - val_loss: 0.5802
Epoch 7/20
215/215 ————— 2s 9ms/step - loss: 0.5838 - val_loss: 0.5796
Epoch 8/20
215/215 ————— 2s 9ms/step - loss: 0.5836 - val_loss: 0.5792
Epoch 9/20
215/215 ————— 2s 9ms/step - loss: 0.5835 - val_loss: 0.5790
Epoch 10/20
215/215 ————— 2s 9ms/step - loss: 0.5824 - val_loss: 0.5790
Epoch 11/20
215/215 ————— 2s 9ms/step - loss: 0.5832 - val_loss: 0.5786
Epoch 12/20
215/215 ————— 2s 9ms/step - loss: 0.5828 - val_loss: 0.5782
Epoch 13/20
215/215 ————— 2s 9ms/step - loss: 0.5817 - val_loss: 0.5781
Epoch 14/20
215/215 ————— 2s 9ms/step - loss: 0.5825 - val_loss: 0.5780
Epoch 15/20
215/215 ————— 2s 10ms/step - loss: 0.5818 - val_loss: 0.5777
Epoch 16/20
215/215 ————— 2s 9ms/step - loss: 0.5819 - val_loss: 0.5778
Epoch 17/20
215/215 ————— 2s 9ms/step - loss: 0.5822 - val_loss: 0.5776

```

```

In [6]: # 4. Visualize the reconstructed images
# Reconstruct some images
decoded_imgs = autoencoder.predict(X_test[:10])

n = 10
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(X_test[i].reshape(28, 28) * 255.0, cmap = "gray")
    plt.axis("off")

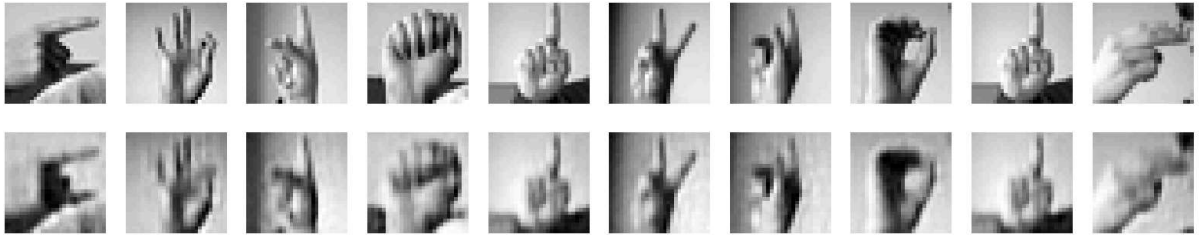
    # Reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28) * 255.0, cmap = "gray")
    plt.axis("off")
plt.show()

```

```

1/1 ————— 1s 638ms/step

```

```
In [7]: # 4. Visualize the PCA-projection of the latent space. Try to include the labels in
# the distribution of the latent space.
# Extract the encoder output (Latent representations)
X_test_encoded = encoder.predict(X_test)

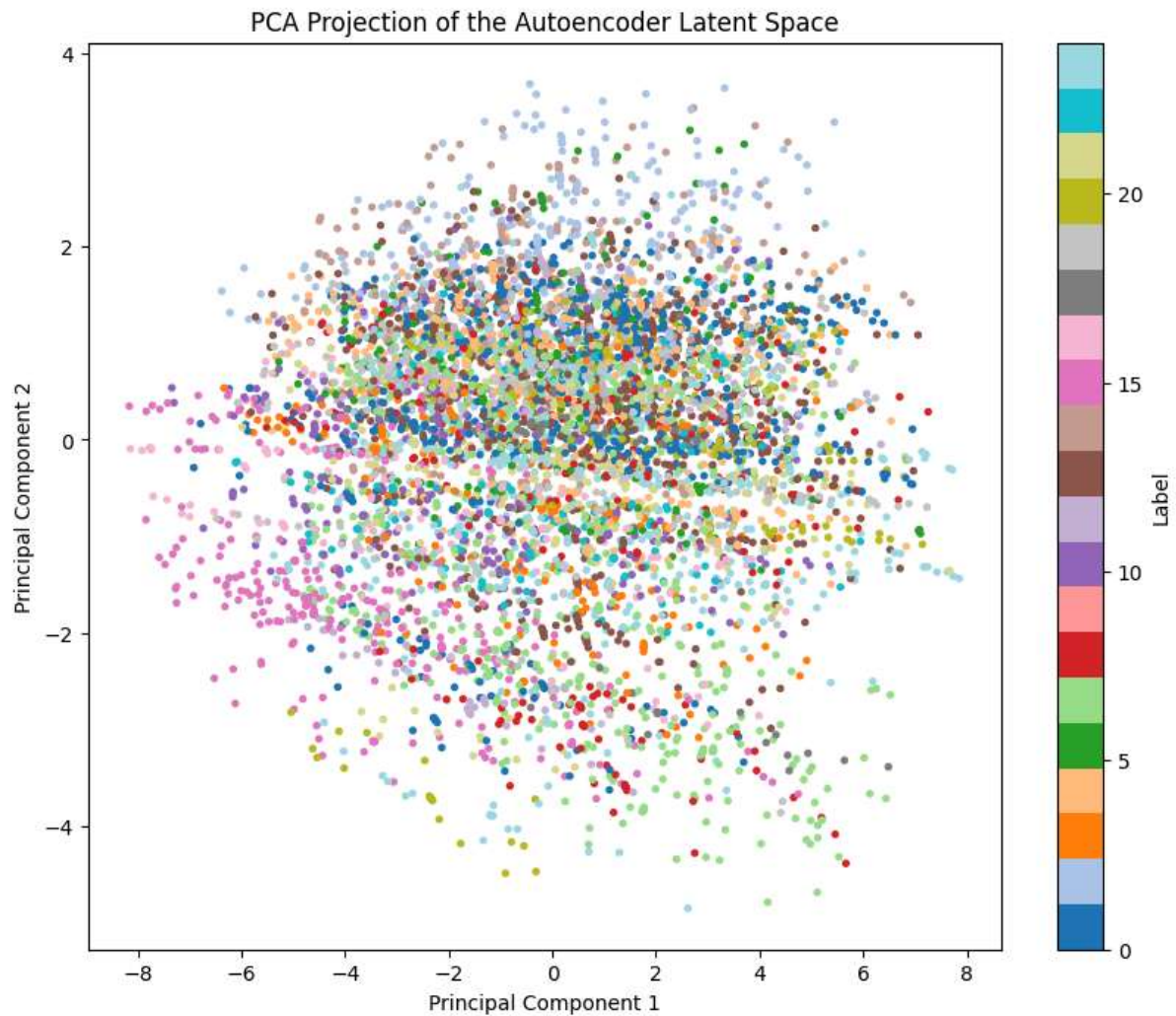
# Flatten the latent feature maps for PCA
X_test_encoded_flat = X_test_encoded.reshape(X_test_encoded.shape[0], -1)

# Apply PCA to reduce to 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_test_encoded_flat)

# Get the true labels for coloring
y_test = test["label"].to_numpy()

# Plot the PCA projection
plt.figure(figsize=(10, 8))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_test, cmap='tab20', s=8)
plt.colorbar(scatter, label='Label')
plt.title("PCA Projection of the Autoencoder Latent Space")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```

225/225 ————— 1s 2ms/step



Discuss here

Discussion

1. Why do we normalize the pixel values of the images?

We normalize the pixel values to make them smaller and more consistent for the model to handle.

Images normally have pixel values between 0 and 255, which are pretty large numbers for a neural network.

By dividing them by 255, we change the range to between 0 and 1.

This helps the model train faster and makes it easier for the optimizer to find good weights.

It also prevents issues like large gradients or unstable learning during training.

2. How does the size of the latent space affect the reconstructed images?

The latent space is like the memory or compressed version of the image inside the autoencoder.

If the latent space is very small, the model doesn't have enough room to store all the important details of the image, so when it tries to rebuild it, the result can look blurry or missing details. On the other hand, if the latent space is large, the model keeps more information and the reconstructed images look sharper and closer to the original ones. However, a larger latent space also means less compression. So there's a balance between how much the model compresses and how clear the output images are.

3. Can autoencoders be used for noise reduction? How?

Yes, autoencoders can be used to remove noise from images. The idea is to train the autoencoder with noisy images as the input and the clean versions of those images as the output.

During training, the model learns how to ignore the noise and focus on the main patterns or shapes in the image.

After it's trained, if you give it a new noisy image, it can produce a cleaner version by filtering out the unwanted noise.

This makes autoencoders useful for improving image quality in areas like photography or medical imaging.

Task 2

In contrast to simple autoencoders, which aim to learn a deterministic mapping from input data to a compressed latent space and back, **Variational Autoencoders (VAEs)** introduce a probabilistic approach to the latent space. While simple autoencoders can reconstruct data effectively, they are not inherently capable of generating new, realistic data samples. VAEs, on the other hand, are designed for *both reconstruction and generation* of novel data samples, making them highly useful for generative modeling tasks.

Why VAEs? A simple autoencoder maps input data to a fixed point in the latent space, meaning that each input has a single corresponding latent vector. While this approach works well for compression and reconstruction, it lacks the ability to generalize and generate new data points because the latent space is not continuous or well-structured.

In contrast, a Variational Autoencoder treats the latent space probabilistically. Instead of mapping each input to a single point, it maps inputs to a distribution (typically a Gaussian distribution) in the latent space. This probabilistic treatment allows for the generation of new samples by sampling from the learned distribution, thus enabling the model to create *new*, unseen data points that resemble the training data.

How VAEs Work? In a VAE, the *encoder* maps the input data to two vectors: a *mean vector* and a *variance (or log variance) vector* (in contrast to one as in a traditional AE). These vectors representing a Gaussian distribution in the latent space. Instead of encoding to a fixed point, the model samples a latent vector from this distribution, using these mean and variance parameters. This sampling introduces variability, allowing the VAE to explore different parts of the latent space and generate diverse outputs.

The *decoder* then reconstructs the data by decoding from the sampled latent vector, which can either come from the encoded input or be randomly generated. This process makes VAEs ideal for generating novel data samples in applications like image generation.

The loss function? The loss function plays a crucial role in training the model to both reconstruct input data and ensure that the latent space is properly structured. The VAE loss function consists of two main components:

1. Reconstruction Loss

Measures how well the VAE is able to reconstruct the input data from the latent space. Typically, this is computed using **binary cross-entropy** (for normalized image data) or **mean squared error** (for continuous data).

For images, binary cross-entropy is often used.

$$\mathcal{L}_{\text{reconstruction}}(x, \hat{x})$$

Where:

- x_i is the original data point (e.g., pixel value)
 - $\hat{x}_i$ is the reconstructed data point
 - N is the number of data points or pixels
-

2. KL Divergence Loss

Ensures that the learned latent distribution $q(z|x)$, parameterized by μ and σ^2 , is close to a standard normal distribution $p(z) = \mathcal{N}(0, I)$.

$$\mathcal{L}_{\text{KL}}(q(z|$$

Where:

- μ_j is the mean of the latent variable distribution

- σ_j^2 is the variance of the latent variable distribution
- d is the dimensionality of the latent space

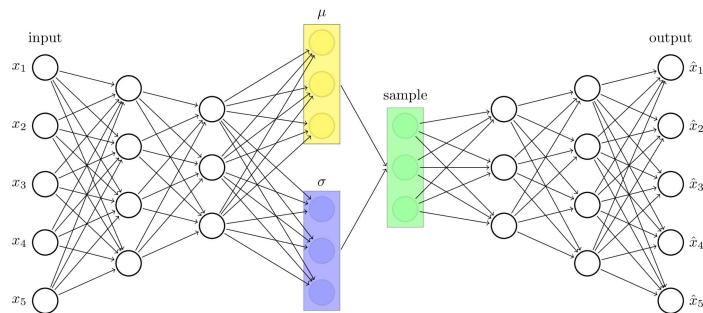
Total VAE Loss

The total loss function for a VAE is the sum of the reconstruction loss and the KL divergence loss:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{reconstruction}} + \mathcal{L}_{\text{KL}}$$

The Role of KL Divergence A crucial aspect of training VAEs is ensuring that the latent space is well-structured and that the learned distributions align with a *prior distribution* (typically a standard normal distribution). This is where the *Kullback-Leibler (KL) divergence loss* comes in. KL divergence measures how different the learned distribution is from the desired prior distribution. By minimizing KL divergence during training, the VAE ensures that the latent space follows a continuous, smooth distribution, making it easier to generate realistic new samples by sampling from this space.

In summary, VAEs combine the strengths of autoencoders for reconstruction with a probabilistic latent space, enabling the generation of new data samples and ensuring smooth interpolation within the latent space through the use of KL divergence. This combination makes VAEs a powerful generative model in unsupervised learning tasks.



Run the following code

```
In [23]: import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras import layers, Model, backend as K
import matplotlib.pyplot as plt
import keras
from keras import ops, layers
from tensorflow.keras.layers import Layer
from sklearn.decomposition import PCA
```

```
In [24]: # Load the MNIST dataset.
train_df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/CA4/sign_mnist_tr
test_df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/CA4/sign_mnist_te
```

```

x_train = train_df.drop(columns=['label']).to_numpy().astype('float32') / 255.0
x_test  = test_df.drop(columns=['label']).to_numpy().astype('float32') / 255.0

x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test  = np.reshape(x_test,  (len(x_test), 28, 28, 1))

y_train = train_df['label'].to_numpy()
y_test  = test_df['label'].to_numpy()

```

```

In [25]: latent_dim = 8 # Dimensionality of the Latent space (small for simplicity)
        input_shape = (28, 28, 1) # Input image shape

```

```

In [26]: # We define the encoder in a similar way as the autoencoder
        input_img = layers.Input(shape=input_shape)
        x = layers.Conv2D(32, 3, activation='relu', padding='same')(input_img)
        x = layers.MaxPooling2D(2, padding='same')(x)
        x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D(2, padding='same')(x)
        x = layers.Flatten()(x)
        x = layers.Dense(128, activation='relu')(x)

```

```

In [27]: # However, instead of a single output layer to the latent space, we have two out
        # (Latent space parameters: mean (mu) and log variance (log_var))
        z_mean = layers.Dense(latent_dim)(x)
        z_log_var = layers.Dense(latent_dim)(x)

```

```

In [28]: # We define a sampling function to sample from the latent space, which introduce
        # (as it still allows gradients to flow backwards)
        # Sampling function
        def sampling(args):
            z_mean, z_log_var = args
            epsilon = K.random_normal(shape=K.shape(z_mean))
            return z_mean + K.exp(0.5 * z_log_var) * epsilon

```

```

In [29]: # There are multiple ways to sample from the latent space, here I introduce a Lambda
        # which allows to use arbitrary functions as Layers in the model
        # Note the output shape of the Lambda Layer; we are returning the (sampled) latent
        # and the learned parameters (mean and log variance)- these are used in the loss
        z = layers.Lambda(sampling, output_shape=(latent_dim,))([z_mean, z_log_var])

```

```

In [30]: # The decoder is defined in the same way as the autoencoder
        decoder_input = layers.Input(shape=(latent_dim,))
        d = layers.Dense(7 * 7 * 64, activation='relu')(decoder_input)
        d = layers.Reshape((7, 7, 64))(d)
        d = layers.Conv2DTranspose(64, 3, activation='relu', strides=2, padding='same')(d)
        d = layers.Conv2DTranspose(32, 3, activation='relu', strides=2, padding='same')(d)
        decoded_img = layers.Conv2DTranspose(1, 3, activation='sigmoid', padding='same')(d)

```

In a Variational Autoencoder (VAE), adjusting the weight of the KL divergence loss controls the balance between data reconstruction accuracy and latent space regularization. Increasing the weight encourages a more structured and continuous latent

space but may reduce reconstruction quality, while decreasing it improves reconstruction but risks losing meaningful structure in the latent space.

```
In [31]: #KL divergence loss weight. This is a hyperparameter that can be tuned, run ALL
# This is the value you should experiment with
KL_weight = 1
```

```
In [32]: # Having build our layers, we can now define the models
encoder = Model(input_img, [z_mean, z_log_var, z], name='encoder')
decoder = Model(decoder_input, decoded_img, name='decoder')

# VAE model that connects the encoder and decoder
vae_output = decoder(encoder(input_img)[2])
vae = Model(input_img, vae_output, name='vae')
```

```
In [33]: # VAE Loss Layer
# Variables: x (input), x_hat (reconstruction), mu (z_mean), logvar (z_log_var)

class VAE_LossLayer(Layer):
    def __init__(self, beta=1.0, rec_kind="bce", **kwargs):
        super(VAE_LossLayer, self).__init__(**kwargs)
        self.beta = beta
        self.rec_kind = rec_kind

    def call(self, inputs):
        x, x_hat, mu, logvar = inputs

        # Reconstruction Loss
        if self.rec_kind == "bce":
            rec_loss = -tf.reduce_sum(
                x * tf.math.log(tf.clip_by_value(x_hat, 1e-7, 1.0)) +
                (1 - x) * tf.math.log(tf.clip_by_value(1 - x_hat, 1e-7, 1.0)),
                axis=[1, 2, 3]
            )
        elif self.rec_kind == "mse":
            rec_loss = tf.reduce_sum(tf.square(x - x_hat), axis=[1, 2, 3])
        else:
            raise ValueError("rec_kind must be 'bce' or 'mse'")

        # KL Divergence
        kl_loss = -0.5 * tf.reduce_sum(1 + logvar - tf.square(mu) - tf.exp(logvar),
            axis=[1, 2, 3])

        # Total Loss
        total_loss = tf.reduce_mean(rec_loss + self.beta * kl_loss)

        self.add_loss(total_loss)
        return x_hat
```

```
In [34]: #Train the VAE

lossed_output = VAE_LossLayer(beta=1.0, rec_kind="bce")(
    [input_img, vae_output, z_mean, z_log_var]
)
```



```
vae = keras.Model(inputs=input_img, outputs=losses_output, name="vae")

vae.compile(optimizer="adam")

#Feel free to play around with the parameters
history = vae.fit(
    x_train,
    epochs=25,
    batch_size=128,
    validation_data=(x_test,),
    verbose=2,
)
```



```

Epoch 1/25
215/215 - 13s - 63ms/step - loss: 506.5787 - val_loss: 494.9826
Epoch 2/25
215/215 - 2s - 10ms/step - loss: 494.9396 - val_loss: 488.5381
Epoch 3/25
215/215 - 2s - 10ms/step - loss: 491.1259 - val_loss: 487.3640
Epoch 4/25
215/215 - 2s - 11ms/step - loss: 488.7147 - val_loss: 484.4233
Epoch 5/25
215/215 - 2s - 10ms/step - loss: 486.8447 - val_loss: 483.1282
Epoch 6/25
215/215 - 2s - 10ms/step - loss: 485.7403 - val_loss: 482.1645
Epoch 7/25
215/215 - 2s - 9ms/step - loss: 484.5858 - val_loss: 481.3779
Epoch 8/25
215/215 - 2s - 9ms/step - loss: 483.9339 - val_loss: 481.0765
Epoch 9/25
215/215 - 2s - 10ms/step - loss: 483.3938 - val_loss: 480.7279
Epoch 10/25
215/215 - 2s - 11ms/step - loss: 482.9623 - val_loss: 480.5002
Epoch 11/25
215/215 - 2s - 10ms/step - loss: 482.5826 - val_loss: 480.5042
Epoch 12/25
215/215 - 2s - 10ms/step - loss: 482.2059 - val_loss: 480.3580
Epoch 13/25
215/215 - 2s - 10ms/step - loss: 481.9059 - val_loss: 480.4139
Epoch 14/25
215/215 - 2s - 10ms/step - loss: 481.7395 - val_loss: 480.3200
Epoch 15/25
215/215 - 2s - 10ms/step - loss: 481.4764 - val_loss: 479.9676
Epoch 16/25
215/215 - 2s - 11ms/step - loss: 481.2122 - val_loss: 480.3494
Epoch 17/25
215/215 - 2s - 10ms/step - loss: 481.0058 - val_loss: 480.1441
Epoch 18/25
215/215 - 2s - 10ms/step - loss: 480.7870 - val_loss: 479.8957
Epoch 19/25
215/215 - 2s - 10ms/step - loss: 480.6219 - val_loss: 480.1789
Epoch 20/25
215/215 - 2s - 10ms/step - loss: 480.4979 - val_loss: 480.0689
Epoch 21/25
215/215 - 2s - 10ms/step - loss: 480.3082 - val_loss: 479.8902
Epoch 22/25
215/215 - 2s - 11ms/step - loss: 480.0954 - val_loss: 480.0488
Epoch 23/25
215/215 - 3s - 14ms/step - loss: 480.0768 - val_loss: 480.1821
Epoch 24/25
215/215 - 2s - 10ms/step - loss: 479.9163 - val_loss: 480.0789
Epoch 25/25
215/215 - 2s - 10ms/step - loss: 479.7523 - val_loss: 480.0620

```

```

In [35]: # Run this visualisation code after training the VAE to answer the discussion q
def plot_latent_space(decoder, n=15, figsize=15):
    """Plots a grid of digits decoded from the VAE's latent space."""
    scale = 2.0
    figure = np.zeros((28 * n, 28 * n))

```

```
grid_x = np.linspace(-scale, scale, n)
grid_y = np.linspace(-scale, scale, n)

pca = PCA(n_components=2)
z_mean, _, _ = encoder.predict(x_test)
z_pca = pca.fit_transform(z_mean)

# Create grid in PCA space
grid_x = np.linspace(-3, 3, n)
grid_y = np.linspace(-3, 3, n)
figure = np.zeros((28 * n, 28 * n))

for i, yi in enumerate(grid_y):
    for j, xi in enumerate(grid_x):
        z_pca_point = np.array([[xi, yi]])
        z_original = pca.inverse_transform(z_pca_point)
        x_decoded = decoder.predict(z_original)
        digit = x_decoded[0].reshape(28, 28)
        figure[i * 28:(i + 1) * 28, j * 28:(j + 1) * 28] = digit

plt.figure(figsize=(figsize, figsize))
plt.imshow(figure, cmap='Greys_r')
plt.show()

# Plot Latent space
plot_latent_space(decoder)
```

225/225 ————— 1s 4ms/step

1/1	—————	0s 317ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 27ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 26ms/step
1/1	—————	0s 27ms/step
1/1	—————	0s 27ms/step
1/1	—————	0s 29ms/step
1/1	—————	0s 29ms/step
1/1	—————	0s 27ms/step
1/1	—————	0s 27ms/step
1/1	—————	0s 26ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 40ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 26ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 26ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 27ms/step
1/1	—————	0s 28ms/step
1/1	—————	0s 27ms/step
1/1	—————	0s 42ms/step
1/1	—————	0s 43ms/step
1/1	—————	0s 44ms/step
1/1	—————	0s 46ms/step
1/1	—————	0s 42ms/step
1/1	—————	0s 42ms/step
1/1	—————	0s 49ms/step
1/1	—————	0s 45ms/step
1/1	—————	0s 44ms/step
1/1	—————	0s 42ms/step
1/1	—————	0s 43ms/step
1/1	—————	0s 41ms/step
1/1	—————	0s 38ms/step
1/1	—————	0s 44ms/step
1/1	—————	0s 57ms/step
1/1	—————	0s 38ms/step
1/1	—————	0s 42ms/step
1/1	—————	0s 39ms/step
1/1	—————	0s 39ms/step
1/1	—————	0s 41ms/step
1/1	—————	0s 39ms/step
1/1	—————	0s 45ms/step
1/1	—————	0s 46ms/step
1/1	—————	0s 48ms/step
1/1	—————	0s 41ms/step
1/1	—————	0s 50ms/step
1/1	—————	0s 51ms/step
1/1	—————	0s 41ms/step
1/1	—————	0s 45ms/step
1/1	—————	0s 44ms/step

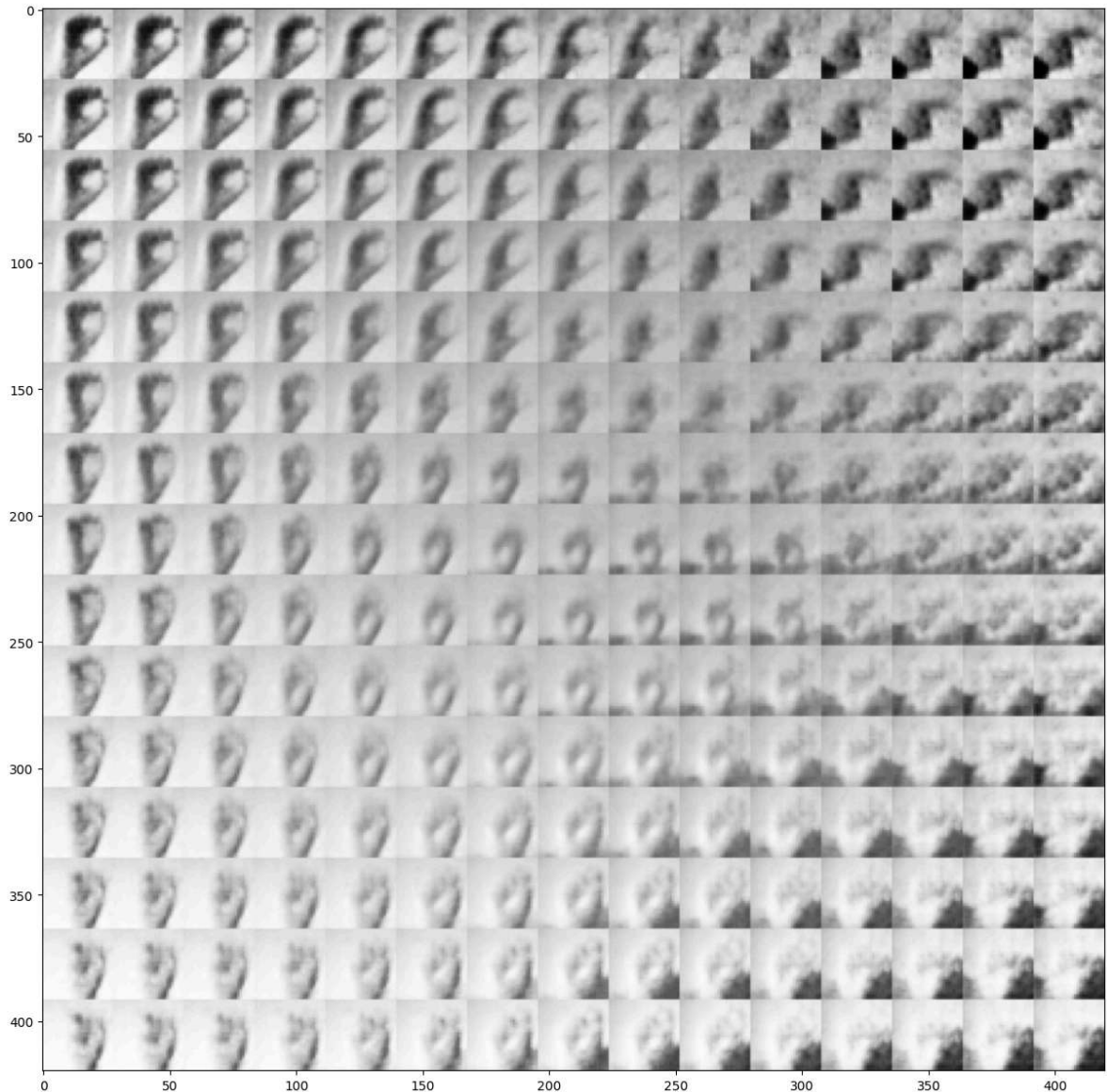
1/1	0s	27ms/step
1/1	0s	28ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	26ms/step
1/1	0s	35ms/step
1/1	0s	30ms/step
1/1	0s	28ms/step
1/1	0s	27ms/step
1/1	0s	29ms/step
1/1	0s	41ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	27ms/step
1/1	0s	28ms/step
1/1	0s	26ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	29ms/step
1/1	0s	33ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	37ms/step
1/1	0s	34ms/step
1/1	0s	32ms/step
1/1	0s	30ms/step
1/1	0s	31ms/step
1/1	0s	31ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	28ms/step
1/1	0s	27ms/step
1/1	0s	31ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	28ms/step
1/1	0s	33ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	28ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	40ms/step
1/1	0s	27ms/step
1/1	0s	30ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step

1/1	0s	30ms/step
1/1	0s	34ms/step
1/1	0s	31ms/step
1/1	0s	31ms/step
1/1	0s	29ms/step
1/1	0s	36ms/step
1/1	0s	31ms/step
1/1	0s	31ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	31ms/step
1/1	0s	38ms/step
1/1	0s	30ms/step
1/1	0s	33ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	30ms/step
1/1	0s	27ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	30ms/step
1/1	0s	28ms/step
1/1	0s	27ms/step
1/1	0s	32ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	32ms/step
1/1	0s	30ms/step
1/1	0s	31ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	27ms/step
1/1	0s	30ms/step
1/1	0s	33ms/step
1/1	0s	30ms/step
1/1	0s	32ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	41ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	27ms/step
1/1	0s	30ms/step

1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	30ms/step
1/1	0s	28ms/step
1/1	0s	27ms/step
1/1	0s	28ms/step
1/1	0s	31ms/step
1/1	0s	28ms/step
1/1	0s	31ms/step
1/1	0s	27ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	32ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	30ms/step
1/1	0s	32ms/step
1/1	0s	27ms/step
1/1	0s	32ms/step
1/1	0s	35ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	36ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	29ms/step
1/1	0s	28ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	31ms/step
1/1	0s	28ms/step
1/1	0s	28ms/step
1/1	0s	29ms/step
1/1	0s	30ms/step
1/1	0s	29ms/step
1/1	0s	35ms/step
1/1	0s	36ms/step
1/1	0s	43ms/step
1/1	0s	43ms/step
1/1	0s	49ms/step
1/1	0s	40ms/step
1/1	0s	40ms/step
1/1	0s	39ms/step
1/1	0s	42ms/step
1/1	0s	47ms/step
1/1	0s	54ms/step
1/1	0s	41ms/step

1/1 ————— 0s 40ms/step

1/1 ————— 0s 39ms/step



Experiment Instructions

Tips: Refer to the visualization above and the loss output from your training cell (make sure `verbose` is set to 1 or 2) to answer the questions.

1. **Adjust the weight of the KL divergence term** (`beta`) in the VAE loss function (e.g., set it to **0.1**, **1**, and **5**).

Run the model with each weight setting and observe the results.

2. **Change the dimensionality of the latent space** (`latent_dim`) to explore how it affects the model's behavior.

Try setting `latent_dim` to **2**, **4**, and **8**.

Re-run your model for each setting and visualize:

- The reconstructed images

- The latent space projection (for 2D, plot directly; for higher dimensions, use PCA or t-SNE)

3. Discussion:

Based on your observations:

- How does changing the KL weight (`beta`) affect the balance between reconstruction quality and latent space smoothness?
- How does increasing the latent space dimensionality (`latent_dim`) affect the expressiveness of the model and the clarity of the reconstructions?
- Were these effects consistent with your expectations? Why or why not?

1. When `KL_weight` was 0.1, the images looked sharp and detailed. When weight was 1, it seemed more balanced. The reconstructions were decent. When we set `KL_weight` to 5, the latent space became smoother and more regular, but the images got blurry, maybe because the model focused more about regularization than accuracy.

2. When we changed the latent dimension, it affected how detailed the images were. With only 2 dimensions, it was easy to plot and understand, but the reconstructions looked worse and the model didn't show all the details. With 4 dimensions, the reconstructions got a bit better and still made some sense. With 8 dimensions, the model could make much better reconstructions. We used PCA to reduce dimension pack to 2.

3. Increasing `KL_weight` made the reconstructions blurrier, while increasing the latent dimension made the images clearer but harder to interpret. Overall, these results matched what we expected because higher `KL_weight` pushes for a more regular latent space, and higher dimensions give the model more freedom to show details.
